

FLUTTER · APRESENTAÇÃO

Racha App

Telas de Login e Cadastro
Código completo e explicado

Arquivo: `apresentacao_auth.dart` · 571 linhas

Três telas · Flutter puro, sem pacotes externos

Como usar este material

Este material tem o **código completo** das três telas e a explicação de cada parte, lado a lado. Serve para estudar sem precisar de computador — no celular mesmo.

A ordem aqui é a mesma do arquivo. Cada parte mostra primeiro o código como ele está, e logo abaixo o que aquele trecho faz e os pontos que costumam virar pergunta.

O ESSENCIAL

Como rodar

Num projeto Flutter novo, cole o conteúdo por cima de `lib/main.dart` e aperte o play. Não precisa mexer no `pubspec.yaml` nem rodar `flutter pub get`.

Por que não precisa de pacote

A única importação do arquivo inteiro é `package:flutter/material.dart`, que já vem em todo projeto Flutter. Não há Firebase, banco de dados nem internet.

O que o `material.dart` traz

Todos os widgets usados aqui: `Scaffold`, `AppBar`, `Column`, `Row`, `TextFormField`, `FilledButton`, `Navigator` e o resto.

O TOPO DO ARQUIVO

```
// =====
// RACHA APP – TELAS DE LOGIN E CADASTRO
// =====
//
// Arquivo único e executável. Duas formas de rodar:
//
// 1. Num projeto Flutter novo, cole este arquivo por cima de lib/main.dart
//    e rode normal (o botão de play do IntelliJ). É o caminho mais simples.
// 2. Dentro do projeto racha_app, onde ele convive com o app de verdade:
//    flutter run -t lib/apresentacao_auth.dart
//
// Só Flutter puro. A única importação é `package:flutter/material.dart` –
// nenhum pacote externo, nenhum banco de dados, nenhuma internet. Por isso
// não é preciso mexer no pubspec.yaml nem rodar `flutter pub get`.
//
// O QUE TEM AQUI:
// - Três telas (login, cadastro e uma tela inicial)
// - Formulários com validação (Form, TextFormField, validator)
// - Navegação entre telas (Navigator.push, pop e pushReplacement)
// - Estado de tela com StatefulWidget e setState
//
// Os usuários cadastrados ficam numa lista na memória do aparelho. Some tudo
// quando o app fecha – é só para as telas funcionarem de verdade durante a
// apresentação, e está explicado na PARTE 2.
// =====

import 'package:flutter/material';
```

Atenção ao estudar: o código abaixo já vem cheio de comentários explicando cada trecho. Eles fazem parte do arquivo e valem tanto quanto a explicação em prosa — vale ler os dois.

Ponto de Entrada

`main()` é a primeira função que roda em qualquer programa Dart. `runApp` recebe um widget e o coloca na tela como raiz de tudo.

O `MaterialApp` envolve o app inteiro e fornece o que vale para todas as telas: o tema, o título e a navegação. Sem ele, `Navigator.push` nem funcionaria.

O CÓDIGO

```
// =====  
// PARTE 1 – PONTO DE ENTRADA  
// =====  
  
void main() {  
  runApp(const RachaApp());  
}  
  
class RachaApp extends StatelessWidget {  
  const RachaApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Racha App',  
  
      // Tira a faixa vermelha de "DEBUG" do canto da tela.  
      debugShowCheckedModeBanner: false,  
  
      theme: ThemeData(  
        useMaterial3: true,  
  
        // A partir de UMA cor, o Flutter gera a paleta inteira do app  
        // (botões, campos, textos). O verde é o do racha.  
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF2E7D32)),  
  
        // Faz TODOS os campos de texto do app terem borda, sem precisar  
        // repetir isso em cada TextFormField.  
        inputDecorationTheme: const InputDecorationTheme(  
          border: OutlineInputBorder(),  
        ),  
      ),  
    );  
  
    // A primeira tela que aparece quando o app abre.  
    home: const LoginScreen(),  
  );  
}
```

PONTOS QUE PODEM SER PERGUNTADOS

Por que `StatelessWidget` aqui

O `RachaApp` só desenha e nunca muda depois de criado — não tem nada para guardar. Quando algo na tela muda, aí sim usa-se `StatefulWidget` (Partes 3 e 4).

`ColorScheme.fromSeed`

A partir de **uma única cor**, o Flutter gera a paleta inteira do app: botões, campos, textos, todos combinando. É por isso que em nenhum lugar do arquivo se escreve a cor de um botão.

`inputDecorationTheme`

Faz **todos** os campos de texto terem borda. Sem isso, seria preciso repetir `border: OutlineInputBorder()` nos sete `TextFormField` do arquivo.

`home:`

Define a primeira tela que aparece quando o app abre — aqui, a `LoginScreen`.

`Theme.of(context)`

Usado nas telas para pegar uma cor ou um tamanho de texto já definido aqui, em vez de escolher um número na mão. Ele sobe pela árvore de widgets até achar este `MaterialApp`. Mudar a cor aqui muda o app inteiro de uma vez.

O Modelo e a Lista de Usuários

Esta parte é Dart puro — nada de Flutter. A classe `Usuario` mantém os dados juntos, para não ficar passando nome, email, idade e peso soltos de uma tela para outra.

A lista `usuariosCadastrados` é onde os cadastros ficam guardados, e as três funções abaixo dela são o que as telas usam para consultar e formatar esses dados.

O CÓDIGO

```
// =====  
// PARTE 2 – O MODELO E A LISTA DE USUÁRIOS  
// =====  
  
/// Um jogador cadastrado no app.  
///  
/// É uma classe comum de Dart: só guarda os dados juntos, para não ficar  
/// passando nome, email, idade e peso soltos de uma tela para outra.  
class Usuario {  
  final String nome;  
  final String email;  
  final String senha;  
  final int idade;  
  final double peso;  
  
  const Usuario({  
    required this.nome,  
    required this.email,  
    required this.senha,  
    required this.idade,  
    required this.peso,  
  });  
}  
  
/// "Banco de dados" da apresentação: uma lista guardada na memória do app.  
///  
/// ATENÇÃO – duas coisas que valem saber explicar, porque podem ser  
/// perguntadas:  
///  
/// 1. A lista some quando o app fecha. Não existe banco de dados nem  
///    servidor aqui; isso é o assunto de uma etapa seguinte do projeto.  
/// 2. A senha está guardada como texto puro. Num app de verdade isso nunca  
///    se faz – a senha vai para um serviço de autenticação que guarda só um  
///    hash dela. Aqui é assim para a tela de login poder ser demonstrada  
///    sem depender de internet.  
final List<Usuario> usuariosCadastrados = [];  
  
/// Procura um usuário que tenha esse email E essa senha.  
/// Devolve `null` quando não acha – é o que a tela de login usa para saber se  
/// deixa entrar ou mostra a mensagem de erro.  
Usuario? procurarUsuario(String email, String senha) {  
  for (final usuario in usuariosCadastrados) {  
    if (usuario.email == email && usuario.senha == senha) {  
      return usuario;  
    }  
  }  
}
```

```

    return null;
}

/// Diz se já existe alguém cadastrado com esse email. Usado pela tela de
/// cadastro para não criar duas contas iguais.
bool emailJaCadastrado(String email) {
  for (final usuario in usuariosCadastrados) {
    if (usuario.email == email) {
      return true;
    }
  }
  return false;
}

/// Deixa a primeira letra de cada palavra maiúscula e o resto minúscula –
/// "gustavo dias" vira "Gustavo Dias". Serve para o nome aparecer sempre do
/// mesmo jeito, independente de como a pessoa digitou.
String capitalizarNome(String nome) {
  return nome
    .trim()
    .split(' ')
    .where((palavra) => palavra.isNotEmpty)
    .map((palavra) =>
      palavra[0].toUpperCase() + palavra.substring(1).toLowerCase())
    .join(' ');
}

```

PONTOS QUE PODEM SER PERGUNTADOS

Campos `final` e `required`

`final` significa que o valor não muda depois de criado. `required` obriga quem cria um `Usuario` a informar todos os campos — esquecer um vira erro na hora de compilar, não na hora de rodar.

Os dados somem quando o app fecha

A lista vive só na memória do aparelho enquanto o app está aberto. Não existe banco de dados nem servidor — isso é assunto de uma etapa seguinte do projeto. Vale dizer isso de forma direta: mostra que a limitação é conhecida e proposital.

A senha está em texto puro

Num app de verdade isso nunca se faz: a senha vai para um serviço de autenticação que guarda só um *hash* dela, e o app nunca vê a senha. Aqui é assim para a tela de login poder ser demonstrada sem depender de internet.

`procurarUsuario` devolve `null`

Quando não acha ninguém com aquele email e senha. É isso que permite a tela de login escrever `if (usuario == null)` para decidir entre entrar e mostrar erro.

`emailJaCadastrado`

É consultada dentro do `validator` do email, na tela de cadastro. Mostra que o `validator` aceita qualquer regra, não só verificar o formato do texto.

Tela de Login

A primeira tela do app. Ela tem dois campos, valida o que foi digitado, procura o usuário na lista da Parte 2 e — se achar — leva para a tela inicial. Se não achar, mostra uma mensagem em vermelho.

É também de onde se chega à tela de cadastro, e para onde o cadastro devolve o usuário recém-criado.

O CÓDIGO

```
// =====
// PARTE 3 – TELA DE LOGIN
// =====

/// É StatefulWidget (e não StatelessWidget) porque esta tela PRECISA GUARDAR COISAS
/// que mudam enquanto ela está aberta: o texto digitado nos campos e a
/// mensagem de erro. Um StatelessWidget não guarda nada.
class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  /// Chave que dá acesso ao estado do Form. É por ela que o botão consegue
  /// mandar validar todos os campos de uma vez só.
  final _formKey = GlobalKey<FormState>();

  /// Cada controller fica "grudado" num campo e permite ler o que foi
  /// digitado nele (`_emailController.text`).
  final _emailController = TextEditingController();
  final _senhaController = TextEditingController();

  /// Mensagem mostrada em vermelho quando o email ou a senha estão errados.
  /// Fica nula enquanto não há erro nenhum.
  String? _erro;

  /// Chamado automaticamente pelo Flutter quando a tela sai de cena.
  /// Os controllers seguram espaço na memória; sem liberá-los aqui, esse
  /// espaço nunca é devolvido (é o que se chama de vazamento de memória).
  @override
  void dispose() {
    _emailController.dispose();
    _senhaController.dispose();
    super.dispose();
  }

  void _entrar() {
    // Dispara o `validator` de TODOS os campos do Form de uma vez.
    // Se algum devolver uma mensagem, o próprio Form mostra o texto em
    // vermelho embaixo do campo e a gente para aqui.
    if (!_formKey.currentState!.validate()) return;

    final usuario = procurarUsuario(
```

```

        _emailController.text.trim().toLowerCase(),
        _senhaController.text,
    );

    if (usuario == null) {
        // setState avisa o Flutter: "algo mudou, desenhe a tela de novo".
        // Sem o setState, a variável até mudaria, mas a tela continuaria
        // mostrando o conteúdo antigo.
        setState(() => _erro = 'Email ou senha inválidos.');
```

return;

```

    }

    // pushReplacement TROCA a tela atual pela nova, em vez de empilhar por
    // cima. É o certo aqui: depois de entrar, o botão "voltar" do Android não
    // deve trazer a pessoa de volta para a tela de login.
    Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (context) => HomeScreen(usuario: usuario)),
    );
}

Future<void> _irParaCadastro() async {
    // push EMPILHA a tela de cadastro por cima desta.
    //
    // O `await` espera essa tela ser fechada. O `<Usuario>` diz que, quando
    // ela fechar, pode vir um Usuario de volta – é o que a tela de cadastro
    // manda no Navigator.pop (ver PARTE 4).
    final novoUsuario = await Navigator.push<Usuario>(
        context,
        MaterialPageRoute(builder: (context) => const CadastroScreen()),
    );

    // Se voltou null, a pessoa apertou o botão de voltar sem se cadastrar.
    if (novoUsuario == null) return;

    // Já preenche o email de quem acabou de se cadastrar, para ela não
    // precisar digitar de novo.
    setState(() {
        _emailController.text = novoUsuario.email;
        _erro = null;
    });
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        // SafeArea afasta o conteúdo do entalhe da câmera e da barra do
        // sistema, para nada ficar escondido embaixo deles.
        body: SafeArea(
            child: Center(
                // Sem o SingleChildScrollView, quando o teclado sobe ele cobre
                // metade da tela e o Flutter reclama que o conteúdo não cabe.
                // Com ele, a tela simplesmente rola.
                child: SingleChildScrollView(
                    padding: const EdgeInsets.all(24),
                    child: Form(
                        key: _formKey,
                        child: Column(
                            // stretch = os filhos ocupam toda a largura disponível.
                            crossAxisAlignment: CrossAxisAlignment.stretch,
                            // min = a Column ocupa só a altura necessária, em vez da
                            // tela inteira. É o que mantém tudo centralizado.
                            mainAxisAlignment: MainAxisAlignment.min,

```

```

children: [
  Icon(
    Icons.sports_soccer,
    size: 64,
    // Pega a cor principal definida lá no tema (PARTE 1),
    // em vez de repetir o código da cor aqui.
    color: Theme.of(context).colorScheme.primary,
  ),
  const SizedBox(height: 8),
  Text(
    'Racha App',
    textAlign: TextAlign.center,
    // Mesma ideia: pega um tamanho de texto já definido no
    // tema, em vez de escolher um número na mão.
    style: Theme.of(context).textTheme.headlineSmall,
  ),

  // SizedBox usado só como espaçador entre os widgets.
  const SizedBox(height: 32),

  TextFormField(
    controller: _emailController,
    // Faz o teclado do celular já abrir com "@" e ".com".
    keyboardType: TextInputType.emailAddress,
    decoration: const InputDecoration(labelText: 'Email'),
    // O validator devolve NULL quando está tudo certo, ou a
    // mensagem de erro quando não está.
    validator: (valor) {
      if (valor == null || !valor.contains('@')) {
        return 'Email inválido';
      }
      return null;
    },
  ),
  const SizedBox(height: 12),

  TextFormField(
    controller: _senhaController,
    // Esconde os caracteres, mostrando bolinhas.
    obscureText: true,
    decoration: const InputDecoration(labelText: 'Senha'),
    validator: (valor) {
      if (valor == null || valor.isEmpty) {
        return 'Informe a senha';
      }
      return null;
    },
  ),

  // Este bloco só entra na tela quando existe erro.
  // O "if" dentro de uma lista de widgets é um recurso do
  // Dart: se a condição for falsa, nada é adicionado.
  if (_erro != null) ...[
    const SizedBox(height: 16),
    Text(
      _erro!,
      textAlign: TextAlign.center,
      style: TextStyle(
        color: Theme.of(context).colorScheme.error,
      ),
    ),
  ],
],

```

```

const SizedBox(height: 24),
FilledButton(
  onPressed: _entrar,
  child: const Text('Entrar'),
),
const SizedBox(height: 12),
TextButton(
  onPressed: _irParaCadastro,
  child: const Text('Não tenho conta – cadastrar'),
),
],
),
),
),
),
),
),
);
}
}

```

PONTOS QUE PODEM SER PERGUNTADOS

Por que StatefulWidget e não StatelessWidget

Porque esta tela precisa **guardar coisas que mudam** enquanto está aberta: o texto digitado nos campos e a mensagem de erro. Um `StatelessWidget` não guarda nada. A regra prática: se algo na tela muda depois que ela apareceu, é `Stateful`.

Por que duas classes

`LoginScreen` é o widget; `_LoginScreenState` guarda o estado. O Flutter separa assim porque o widget é descartado e recriado o tempo todo, enquanto o estado sobrevive.

GlobalKey<FormState>

Dá acesso ao `Form`, para o botão conseguir validar todos os campos de uma vez com `_formKey.currentState!.validate()`.

TextEditingController

Fica grudado num campo e permite ler o que foi digitado, com `_emailController.text`.

O validator

Devolve `null` quando está tudo certo, ou o texto da mensagem quando está errado. O `Form` mostra esse texto em vermelho embaixo do campo, sozinho.

setState — provavelmente a pergunta mais provável

Avisa o Flutter: “alguma coisa mudou, desenhe a tela de novo”. Sem ele, a variável `_erro` até mudaria de valor, mas a tela continuaria mostrando o conteúdo antigo.

dispose

O Flutter chama esse método sozinho quando a tela sai de cena. Os controllers seguram espaço na memória; sem liberá-los aqui, esse espaço nunca é devolvido — é o que se chama de **vazamento de memória**. O `super.dispose()` no fim chama a versão da classe mãe.

O `if` dentro da lista de widgets

`if (_erro != null) ...[ ... ]` só entra na tela quando existe erro. É um recurso do Dart: um `if` dentro de uma lista simplesmente não adiciona nada quando a condição é falsa. Os três pontos espalham vários widgets de uma vez.

`SafeArea` e `SingleChildScrollView`

`SafeArea` afasta o conteúdo do entalhe da câmera e da barra do sistema. `SingleChildScrollView` faz a tela rolar — sem ele, quando o teclado sobe e cobre metade da tela, o Flutter mostra a faixa listrada de “conteúdo não cabe”.

`obscureText: true`

Esconde os caracteres da senha, mostrando bolinhas.

Tela de Cadastro

A tela de criar conta. A estrutura é a mesma do login — `StatefulWidget`, `Form`, `GlobalKey`, `controllers`, `dispose` — só que com cinco campos em vez de dois.

No fim ela adiciona o novo usuário na lista da Parte 2 e fecha, devolvendo esse usuário para a tela de login.

O CÓDIGO

```
// =====
// PARTE 4 – TELA DE CADASTRO
// =====

class CadastroScreen extends StatefulWidget {
  const CadastroScreen({super.key});

  @override
  State<CadastroScreen> createState() => _CadastroScreenState();
}

class _CadastroScreenState extends State<CadastroScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nomeController = TextEditingController();
  final _emailController = TextEditingController();
  final _senhaController = TextEditingController();
  final _idadeController = TextEditingController();
  final _pesoController = TextEditingController();

  @override
  void dispose() {
    _nomeController.dispose();
    _emailController.dispose();
    _senhaController.dispose();
    _idadeController.dispose();
    _pesoController.dispose();
    super.dispose();
  }

  void _cadastrar() {
    if (!_formKey.currentState!.validate()) return;

    final usuario = Usuario(
      nome: capitalizarNome(_nomeController.text),
      email: _emailController.text.trim().toLowerCase(),
      senha: _senhaController.text,
      // Aqui pode usar `parse` (que quebra se o texto não for número)
      // porque o validator do campo JÁ garantiu que é. Ver o comentário
      // sobre parse/tryParse no campo de idade, mais abaixo.
      idade: int.parse(_idadeController.text),
      // Brasileiro digita "72,5"; o Dart só entende "72.5".
      peso: double.parse(_pesoController.text.replaceAll(',', '.')),
    );

    usuariosCadastrados.add(usuario);
  }
}
```

```

// SnackBar é a tarjinha que sobe do rodapé da tela.
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(content: Text('Conta criada! Bem-vindo, ${usuario.nome}.')),
);

// pop FECHA esta tela e volta para a anterior (o login).
//
// O segundo argumento é o que vai ser DEVOLVIDO para quem abriu esta
// tela – é exatamente o valor que o `await Navigator.push<Usuario>` da
// tela de login está esperando (ver PARTE 3).
Navigator.pop(context, usuario);
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    // A AppBar já vem com o botão de voltar sozinha, porque esta tela foi
    // aberta com push (tem uma tela embaixo dela). Esse botão faz um
    // Navigator.pop sem devolver nada – daí o `null` lá na PARTE 3.
    appBar: AppBar(title: const Text('Criar conta')),
    body: SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(24),
        child: Form(
          key: _formKey,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const CircleAvatar(
                radius: 48,
                child: Icon(Icons.person, size: 48),
              ),
              const SizedBox(height: 24),

              TextFormField(
                controller: _nomeController,
                decoration: const InputDecoration(labelText: 'Nome'),
                validator: (valor) {
                  if (valor == null || valor.trim().isEmpty) {
                    return 'Informe o nome';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 12),

              TextFormField(
                controller: _emailController,
                keyboardType: TextInputType.emailAddress,
                decoration: const InputDecoration(labelText: 'Email'),
                validator: (valor) {
                  if (valor == null || !valor.contains('@')) {
                    return 'Email inválido';
                  }
                  // O validator não serve só para o formato: dá para usar
                  // para qualquer regra. Aqui ele impede duas contas com
                  // o mesmo email.
                  if (emailJaCadastrado(valor.trim().toLowerCase())) {
                    return 'Já existe uma conta com esse email';
                  }
                  return null;
                },
              ),
            ],
          ),
        ),
      ),
    ),
  );
}

```

```

const SizedBox(height: 12),

TextFormField(
  controller: _senhaController,
  obscureText: true,
  decoration: const InputDecoration(labelText: 'Senha'),
  validator: (valor) {
    if (valor == null || valor.length < 6) {
      return 'Mínimo de 6 caracteres';
    }
    return null;
  },
),
const SizedBox(height: 12),

// Row coloca os widgets lado a lado (na horizontal).
// Expanded faz cada campo ocupar metade da largura – sem
// ele, os campos tentariam ter largura infinita dentro do
// Row e o app mostraria a faixa amarela e preta de erro.
Row(
  children: [
    Expanded(
      child: TextFormField(
        controller: _idadeController,
        // Teclado só de números.
        keyboardType: TextInputType.number,
        decoration: const InputDecoration(labelText: 'Idade'),
        validator: (valor) {
          // tryParse devolve null quando o texto não é um
          // número, em vez de quebrar o app como o parse
          // faria. É isso que permite usá-lo como teste.
          if (int.tryParse(valor ?? '') == null) {
            return 'Idade inválida';
          }
          return null;
        },
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: TextFormField(
        controller: _pesoController,
        // Teclado de números COM vírgula, diferente do campo
        // de idade.
        keyboardType: const TextInputType.numberWithOptions(
          decimal: true,
        ),
        decoration:
          const InputDecoration(labelText: 'Peso (kg)'),
        validator: (valor) {
          final texto = (valor ?? '').replaceAll(',', '.');
          if (double.tryParse(texto) == null) {
            return 'Peso inválido';
          }
          return null;
        },
      ),
    ),
  ],
),

const SizedBox(height: 24),
FilledButton(

```

```

        onPressed: _cadastrar,
        child: const Text('Cadastrar'),
      ),
    ],
  ),
),
),
),
);
}
}

```

PONTOS QUE PODEM SER PERGUNTADOS

O `validator` não serve só para formato

O campo de email tem **duas** regras: verificar se tem `@` e consultar `emailJaCadastrado` para não criar duas contas iguais. Vale mostrar isso se perguntarem para que serve o `validator`.

`parse` e `tryParse`

`tryParse` devolve `null` quando o texto não é número, em vez de quebrar o app — é isso que permite usá-lo como teste dentro do `validator`. Já o `parse` pode ser usado no `_cadastrar` sem risco, porque naquele ponto o `validator` já garantiu que o texto é um número.

A vírgula do campo de peso

Brasileiro digita `72,5`; o Dart só entende `72.5`. O `replaceAll(',', '.')` aparece **duas vezes** — no `validator` e no `_cadastrar` — e precisa ser igual nos dois, senão o campo passa na validação e quebra na hora de salvar.

Teclados diferentes

Idade usa `TextInputType.number` (só dígitos) e peso usa

`TextInputType.numberWithOptions(decimal: true)` (com vírgula).

`Row` + `Expanded`

`Row` coloca idade e peso lado a lado; cada `Expanded` faz um ocupar metade da largura. **Sem o `Expanded`**, os campos tentariam ter largura infinita dentro do `Row` e apareceria a faixa amarela e preta de overflow. É uma das perguntas mais comuns sobre layout.

`SnackBar`

É a tarjinha que sobe do rodapé da tela. O `ScaffoldMessenger.of(context)` acha o `Scaffold` mais próximo na árvore — mesma ideia do `Theme.of(context)`.

Interpolação de string

O `${usuario.nome}` dentro das aspas coloca o valor da variável no meio do texto.

`Navigator.pop(context, usuario)`

Fecha esta tela e **devolve o usuário** para quem a abriu. Detalhado na Parte 5.

Tela Inicial

A tela que aparece depois de entrar. Ela só mostra os dados do usuário que recebeu pronto da tela de login — por isso é `StatelessWidget`, e não `Stateful`: não há nada nela que mude.

É também onde fica o botão de sair.

O CÓDIGO

```
// =====
// PARTE 5 – TELA INICIAL
// =====

/// Esta é StatelessWidget porque não guarda nada que mude: ela só mostra o
/// usuário que recebeu pronto da tela de login.
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key, required this.usuario});

  /// O dado que veio da tela anterior, passado pelo construtor.
  final Usuario usuario;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Racha App'),
        // A AppBar NÃO mostra botão de voltar aqui, porque esta tela entrou
        // com pushReplacement: não há nada empilhado embaixo dela.
        actions: [
          IconButton(
            icon: const Icon(Icons.logout),
            tooltip: 'Sair',
            // Sair é o caminho inverso: troca esta tela pela de login.
            onPressed: () => Navigator.pushReplacement(
              context,
              MaterialPageRoute(builder: (context) => const LoginScreen()),
            ),
          ),
        ],
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const CircleAvatar(
              radius: 56,
              child: Icon(Icons.person, size: 56),
            ),
            const SizedBox(height: 16),
            Text(
              'Bem-vindo, ${usuario.nome}!',
              style: Theme.of(context).textTheme.titleLarge,
            ),
            const SizedBox(height: 8),
            Text(usuario.email),
            // toStringAsFixed(1) mostra o peso com uma casa decimal:
            // 72.0 em vez de 72.

```

```
        Text('${usuario.idade} anos · ${usuario.peso.toStringAsFixed(1)} kg'),  
      ],  
    ),  
  );  
}  
}
```

PONTOS QUE PODEM SER PERGUNTADOS

Como o dado chega aqui

Pelo **construtor**. O `required this.usuario` obriga quem cria a tela a passar o usuário, e o campo `final Usuario usuario` guarda o valor. É a forma mais simples de passar dados entre telas no Flutter.

Por que não tem botão de voltar

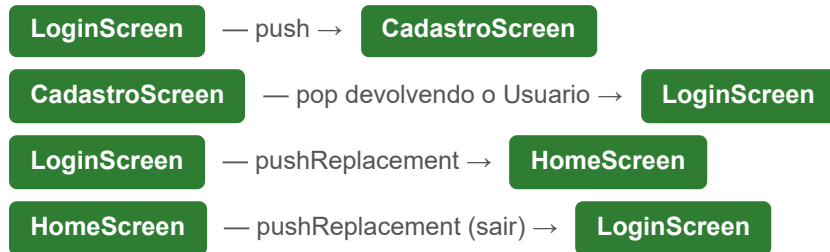
Porque esta tela entrou com `pushReplacement` — não há nada empilhado embaixo dela. A `AppBar` cria esse botão sozinha só quando existe uma tela anterior.

`toStringAsFixed(1)`

Mostra o peso com uma casa decimal: `72.0` em vez de `72`.

A navegação entre as telas

Esta é a parte que o professor pediu especificamente, então vale ser a mais bem estudada. O arquivo usa **três** formas de navegar, e cada escolha tem um motivo.



A pilha de telas

O `Navigator` funciona como uma **pilha**: cada tela nova entra por cima da anterior, e voltar significa tirar a de cima. É por isso que os nomes são `push` (empilhar) e `pop` (desempilhar).

1. `push` — empilhar por cima

```
final novoUsuario = await Navigator.push<Usuario>(
  context,
  MaterialPageRoute(builder: (context) => const CadastroScreen()),
);
```

O `MaterialPageRoute` dá a animação padrão de entrada da tela. O `builder` é uma função que constrói a tela nova — ela só é criada no momento em que a navegação acontece, não antes.

Como o login continua na pilha embaixo do cadastro, a `AppBar` do cadastro ganha o **botão de voltar sozinho**, sem ninguém programá-lo.

2. `pop` devolvendo um dado

Esta é a parte mais interessante do arquivo, porque muita gente não sabe que dá para fazer:

```
// Na tela de cadastro:
Navigator.pop(context, usuario);
```

O segundo argumento do `pop` é **devolvido para quem abriu a tela**. É exatamente o valor que o `await Navigator.push<Usuario>` estava esperando lá no login:

```
// De volta na tela de login:
if (novoUsuario == null) return;
setState(() {
  _emailController.text = novoUsuario.email;
});
```

O `await` faz o código do login **esperar** a tela de cadastro fechar antes de continuar. O `<Usuario>` diz que tipo de valor pode voltar.

E se voltar `null` ? Significa que a pessoa apertou o botão de voltar da `AppBar` em vez de se cadastrar — esse botão faz um `pop` sem devolver nada. Por isso a verificação na primeira linha.

O resultado prático: quem acaba de se cadastrar volta para o login com o email **já preenchido**.

3. `pushReplacement` — trocar em vez de empilhar

```
Navigator.pushReplacement(  
  context,  
  MaterialPageRoute(builder: (context) => HomeScreen(usuario: usuario)),  
);
```

Ele **troca** a tela atual pela nova, em vez de empilhar por cima.

Por que aqui e não `push` ? Porque depois de entrar no app, o botão “voltar” do Android não deve trazer a pessoa de volta para a tela de login. Com `pushReplacement`, o login sai da pilha — e é por isso que a `HomeScreen` não tem botão de voltar na `AppBar`.

O mesmo vale no caminho inverso: o botão de sair também usa `pushReplacement` para voltar ao login.

Perguntas prováveis

As respostas estão curtas de propósito: são para responder em voz alta, não para ler.

Pergunta	Resposta curta
Qual a diferença entre <code>StatelessWidget</code> e <code>StatefulWidget</code> ?	Stateless só desenha e não guarda nada. Stateful guarda coisas que mudam enquanto a tela está aberta. Login e cadastro são Stateful (têm campos digitáveis); a home é Stateless.
Para que serve o <code>setState</code> ?	Avisa o Flutter que algo mudou e a tela precisa ser redesenhada. Sem ele o valor muda, mas a tela continua mostrando o conteúdo antigo.
O que acontece quando clico em Entrar?	O <code>_entrar</code> valida o formulário, chama <code>procurarUsuario</code> na lista, e ou mostra erro com <code>setState</code> ou vai para a home com <code>pushReplacement</code> .
Qual a diferença entre <code>push</code> e <code>pushReplacement</code> ?	<code>push</code> empilha a tela nova por cima (dá para voltar). <code>pushReplacement</code> troca a atual pela nova (não dá para voltar) — usado depois do login para o botão voltar não retornar à tela de senha.
Como um dado volta de uma tela para outra?	<code>Navigator.pop(context, usuario)</code> devolve o valor; quem abriu com <code>await Navigator.push<Usuario></code> recebe. É assim que o email vem preenchido depois do cadastro.
Por que a home não tem botão de voltar?	Porque entrou com <code>pushReplacement</code> — não há tela empilhada embaixo dela. A <code>AppBar</code> cria esse botão sozinha só quando existe.
Para que serve a <code>GlobalKey<FormState></code> ?	Dá acesso ao <code>Form</code> para o botão validar todos os campos de uma vez com <code>_formKey.currentState!.validate()</code> .
O que o <code>validator</code> devolve?	<code>null</code> quando está certo, ou o texto da mensagem quando está errado. O <code>Form</code> mostra esse texto embaixo do campo.
Para que serve o <code>dispose</code> ?	Liberar os <code>TextEditingController</code> quando a tela sai de cena. Sem isso há vazamento de memória.
E se eu digitar email sem @ ?	O <code>validator</code> devolve a mensagem, o <code>Form</code> mostra em vermelho e o método para na primeira linha.
Qual a diferença entre <code>parse</code> e <code>tryParse</code> ?	<code>tryParse</code> devolve <code>null</code> se não for número (usado no <code>validator</code>); <code>parse</code> quebra o app (usado depois, onde o validator já garantiu o formato).
Por que <code>Expanded</code> dentro do <code>Row</code> ?	Para cada campo ocupar metade da largura. Sem ele os campos teriam largura infinita e apareceria a faixa de overflow.
Onde os usuários ficam guardados?	Numa lista na memória (Parte 2). Somem quando o app fecha — banco de dados é a próxima etapa do projeto.

Pergunta	Resposta curta
Por que a senha está visível no código?	Porque não há servidor. Num app real ela vai para um serviço de autenticação que guarda só o hash, e o app nunca vê a senha.
Como os botões ficaram verdes sem ninguém pintar?	Pelo <code>ColorScheme.fromSeed</code> no tema (Parte 1): a partir de uma cor, o Flutter gera a paleta inteira.

Antes da apresentação

Cada pessoa deve conseguir percorrer um caminho inteiro

Não basta decorar uma parte. O mais provável de ser cobrado é: botão Cadastrar → `validate()` → cria o `Usuario` → adiciona na lista → `pop` devolvendo → o login recebe e preenche o email.

Rodem o app antes

Façam o caminho completo: cadastrar, voltar, entrar, sair. Ver a tela funcionando com o código aberto do lado ajuda muito mais a explicar do que reler o arquivo.

Se alguma parte continuar confusa, vale apagar

Código que ninguém do grupo sabe explicar pesa mais contra do que a falta dele.